

TUGAS PENDAHULUAN MODUL 13



Disusun Oleh :

Rengganis Tantri Pramudita (2311104065)

Kelas : SE0702

Dosen :

Yudha Islami Sulistya

PROGRAM STUDI SOFTWARE ENGINEERING

DIREKTORAT KAMPUS PURWOKERTO

TELKOM UNIVERSITY PURWOKERTO

2025

1. MENJELASKAN SALAH SATU DESIGN PATTERN

A. Berikan salah satu contoh kondisi dimana design pattern “Observer” dapat digunakan

Jawab:

Design pattern Observer cocok digunakan ketika terdapat satu objek (subject) yang perubahannya harus diinformasikan secara otomatis ke banyak objek lainnya (observer). Contohnya, Dalam aplikasi cuaca ketika data cuaca (seperti suhu, kelembaban) diperbarui, tampilan antarmuka pengguna (UI) harus ikut diperbarui secara otomatis. Dalam kasus ini, data cuaca adalah *subject*, dan UI adalah *observer*.

B. Berikan penjelasan singkat mengenai langkah-langkah dalam mengimplementasikan design pattern “Observer”

Jawab:

- Buat interface atau abstract class Observer dengan method seperti update().
- Buat interface atau abstract class Subject dengan method seperti attach(observer), detach(observer), dan notifyObservers().
- Kelas ConcreteSubject menyimpan state dan daftar observer. Ketika state berubah, ia akan memanggil notifyObservers() untuk memberi tahu semua observer.
- Kelas ConcreteObserver mengimplementasikan method update() dan bereaksi sesuai dengan perubahan pada subject.

C. Berikan kelebihan dan kekurangan dari design pattern “Observer”

Jawab:

Kelebihan	Kekurangan
Mengurangi <i>tight coupling</i> antara objek subject dan observer.	Dapat menyebabkan masalah performa jika terlalu banyak observer.
Mempermudah pengembangan sistem yang membutuhkan notifikasi otomatis.	Sulit untuk melakukan <i>debugging</i> karena notifikasi terjadi secara tersembunyi.
Memungkinkan untuk menambahkan observer baru tanpa mengubah kode subject.	Tidak menjamin urutan notifikasi jika banyak observer yang terdaftar.

2. IMPLEMENTASI DAN PEMAHAMAN DESIGN PATTERN OBSERVER

Jawab:

a. ConcreateObserverA.cs

```
using System;
namespace tpmodul13_2311104065
{
    class ConcreteObserverA : IObserved
    {
        public void Update(ISubject subject)
        {
            if (subject is Subject concreteSubject && concreteSubject.State < 3)
            {
                Console.WriteLine("ConcreteObserverA: Reacted to the event.");
            }
        }
    }
}
```

b. ConcreateObserverb.cs

```
using System;
namespace tpmodul13_2311104065
{
    class ConcreteObserverB : IObserved
    {
        public void Update(ISubject subject)
        {
            if (subject is Subject concreteSubject)
            {
                if (concreteSubject.State == 0 || concreteSubject.State >= 2)
                {
                    Console.WriteLine("ConcreteObserverB: Reacted to the event.");
                }
            }
        }
    }
}
```

```
    }  
    }  
    }  
    }  
}
```

c. IObserver.cs

```
using System;  
using System.Collections.Generic;  
using System.Linq;  
using System.Runtime.InteropServices.JavaScript;  
using System.Text;  
using System.Threading.Tasks;  
  
namespace tpmodul13_2311104065  
{  
    public interface IObserver  
    {  
        void Update(ISubject subject);  
    }  
}
```

d. ISubject.cs

```
using System;  
using System.Collections.Generic;  
using System.Linq;  
using System.Text;  
using System.Threading.Tasks;  
  
namespace tpmodul13_2311104065
```

```
{  
    public interface ISubject  
    {  
        void Attach(IObserver observer);  
  
        void Detach(IObserver observer);  
  
        void Notify();  
    }  
}
```

e. Program.cs

```
using tpmodul13_2311104065;  
class Program  
{  
    static void Main(string[] args)  
    {  
        var subject = new Subject();  
        var observerA = new ConcreteObserverA();  
        subject.Attach(observerA);  
  
        var observerB = new ConcreteObserverB();  
        subject.Attach(observerB);  
  
        subject.SomeBusinessLogic();  
        subject.SomeBusinessLogic();  
  
        subject.Detach(observerB);  
  
        subject.SomeBusinessLogic();  
    }  
}
```

f. Subject.cs

```
using System;
using System.Collections.Generic;
using System.Threading;
namespace tpmodul13_2311104065
{
    public class Subject : ISubject
    {
        // State of the subject
        public int State { get; set; } = 0;

        // List of subscribers
        private readonly List<IObserver> _observers = [];

        // Attach an observer
        public void Attach(IObserver observer)
        {
            Console.WriteLine("Subject: Attached an observer.");
            _observers.Add(observer);
        }

        // Detach an observer
        public void Detach(IObserver observer)
        {
            _observers.Remove(observer);
            Console.WriteLine("Subject: Detached an observer.");
        }

        // Notify all observers
        public void Notify()
        {

```

```

        Console.WriteLine("Subject: Notifying observers...");

        foreach (var observer in _observers)
        {
            observer.Update(this);
        }
    }

    // Simulate some business logic
    public void SomeBusinessLogic()
    {
        Console.WriteLine("\nSubject: I'm doing something important.");
        State = Random.Shared.Next(0, 10); // Disarankan sejak C# 9.0

        Thread.Sleep(15);

        Console.WriteLine("Subject: My state has just changed to: " + State);
        Notify();
    }
}

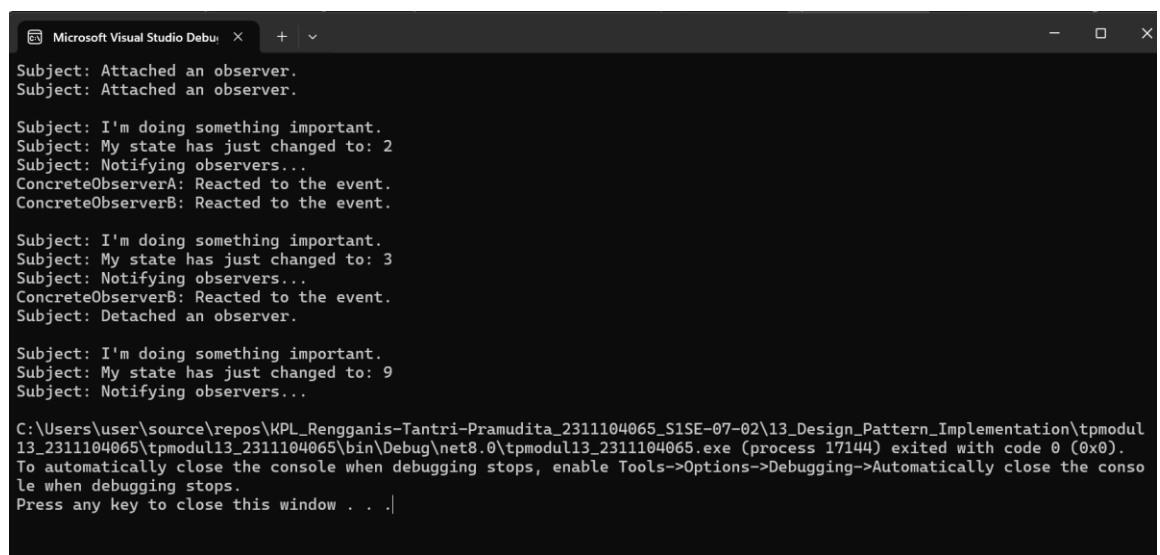
```

Penjelasan:

- `var subject = new Subject();`
Membuat objek subject (subjek yang diamati). Objek ini akan menjadi pusat perubahan status yang akan diberitahukan ke observer.
- `var observerA = new ConcreteObserverA();`
Membuat instance observer pertama (ConcreteObserverA), yaitu class yang akan merespons perubahan dari Subject.
- `subject.Attach(observerA);`
Mendaftarkan observerA ke dalam list observer milik subject, artinya observerA akan menerima notifikasi ketika subject berubah.

- `var observerB = new ConcreteObserverB();`
Membuat instance observer kedua (`ConcreteObserverB`), observer lain yang juga akan bereaksi terhadap perubahan dari subject.
- `subject.Attach(observerB);`
Mendaftarkan observerB ke subject, sehingga sekarang ada dua observer yang akan diberi tahu ketika subject berubah.
- `subject.SomeBusinessLogic();`
Memanggil method yang berisi logika bisnis pada subject. Metode ini akan mengubah State milik subject, lalu memberi tahu semua observer bahwa ada perubahan.
- `subject.SomeBusinessLogic();`
Memanggil lagi method logika bisnis. Sama seperti sebelumnya, akan mengubah State dan men-trigger notifikasi ke observer.
- `subject.Detach(observerB);`
Menghapus observerB dari daftar observer. Setelah ini, observerB tidak akan menerima notifikasi lagi dari subject.
- `subject.SomeBusinessLogic();`
Memanggil method logika bisnis terakhir kalinya. Kali ini hanya observerA yang akan bereaksi, karena observerB sudah dilepas.

Outputnya



```

Microsoft Visual Studio Debug
Subject: Attached an observer.
Subject: Attached an observer.

Subject: I'm doing something important.
Subject: My state has just changed to: 2
Subject: Notifying observers...
ConcreteObserverA: Reacted to the event.
ConcreteObserverB: Reacted to the event.

Subject: I'm doing something important.
Subject: My state has just changed to: 3
Subject: Notifying observers...
ConcreteObserverB: Reacted to the event.
Subject: Detached an observer.

Subject: I'm doing something important.
Subject: My state has just changed to: 9
Subject: Notifying observers...

C:\Users\user\source\repos\KPL_Rengganis-Tantri-Pramudita_2311104065_SISE-07-02\13_Design_Pattern_Implementation\tpmodul13_2311104065\tpmodul13_2311104065\bin\Debug\net8.0\tpmodul13_2311104065.exe (process 17144) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|

```